

RAG infrastructure for generative AI using Vertex AI and AlloyDB for PostgreSQL



Last reviewed 2026-02-04 UTC

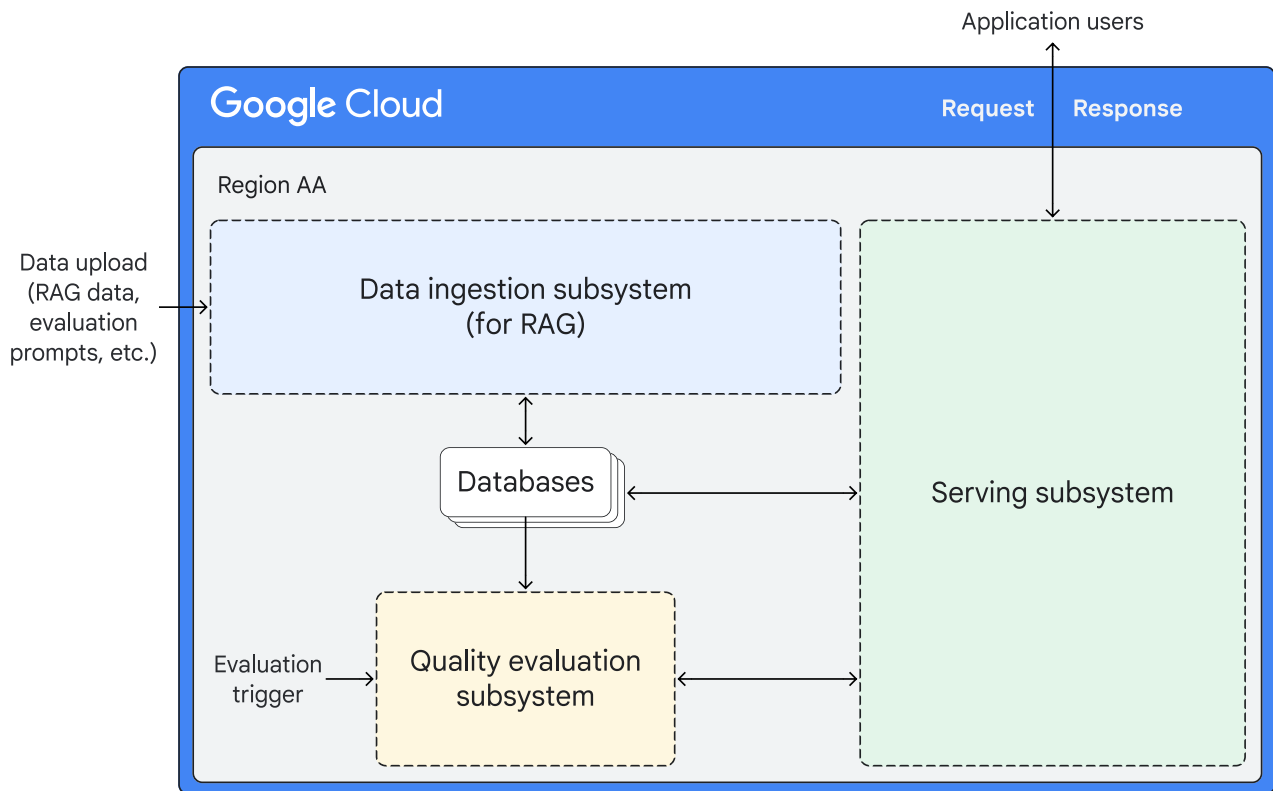
This document provides a reference architecture that you can use to design the infrastructure to run a generative artificial intelligence (AI) application with retrieval-augmented generation (RAG).

(https://developers.google.com/machine-learning/glossary#retrieval-augmented_generation). The intended audience for this document includes developers and administrators of generative AI applications and cloud architects. The document assumes a basic understanding of AI, machine learning (ML), and large language model (LLM) concepts. This document doesn't provide guidance about how to design and develop a generative AI application.

Note: To learn how you can use open source tools like Ray, Hugging Face, and LangChain to rapidly build and deploy a RAG-capable generative AI application on Google Kubernetes Engine (GKE), see [RAG infrastructure for generative AI using GKE and Cloud SQL](#) ([/architecture/rag-capable-gen-ai-app-using-gke](#)).

Architecture

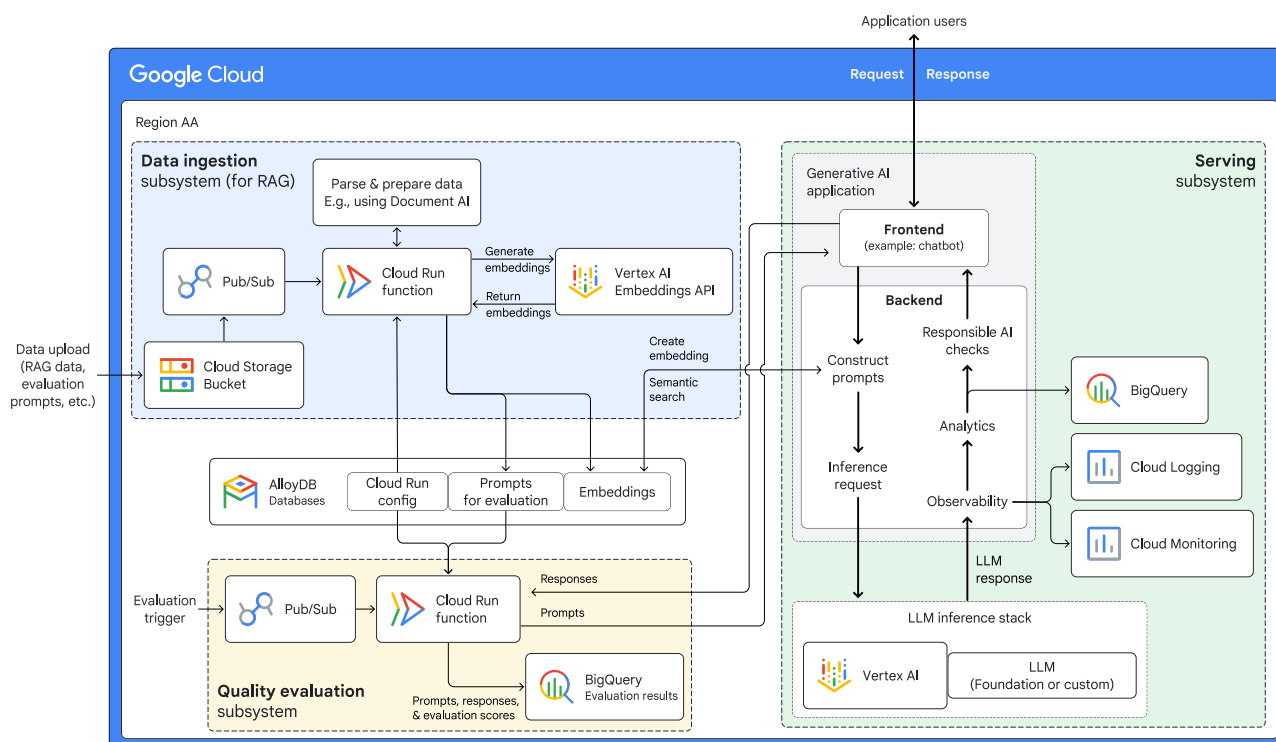
The following diagram shows a high-level view of an architecture for a RAG-capable generative AI application in Google Cloud:



The architecture contains the following interconnected components:

Component	Purpose	Interactions
Data ingestion subsystem	Prepare and process the external data that's used to enable the RAG capability.	The data ingestion subsystem interacts with the other subsystems in the architecture through the database layer.
Serving subsystem	Handle the request-response flow between the generative AI application and its users.	The serving subsystem interacts with the data ingestion subsystem through the database layer.
Quality evaluation subsystem	Evaluate the quality of responses that the serving subsystem generates.	The quality evaluation subsystem interacts with the serving subsystem directly and with the data ingestion subsystem through the database layer.
Databases	Store the following data: - Prompts - Vectorized embeddings of the data used for RAG - Configurations of the serverless functions in the data ingestion and quality evaluation subsystems	All the subsystems in the architecture interact with the databases.

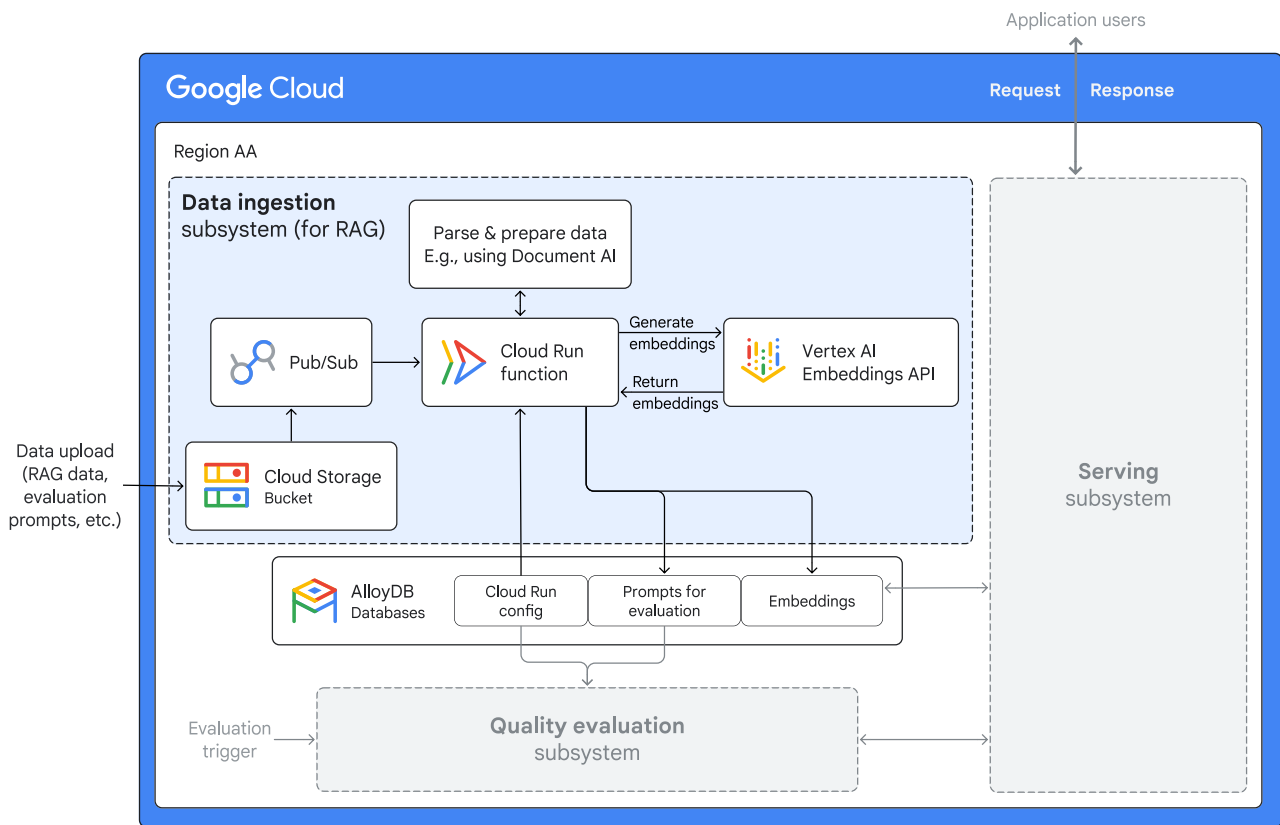
The following diagram shows a detailed view of the architecture:



The following sections provide detailed descriptions of the components and data flow within each subsystem of the architecture.

Data ingestion subsystem

The data ingestion subsystem ingests data from external sources such as files, databases, and streaming services. The uploaded data includes prompts for quality evaluation. The data ingestion subsystem provides the RAG capability in the architecture. The following diagram shows details of the data ingestion subsystem in the architecture:



The following are the steps in the data-ingestion flow:

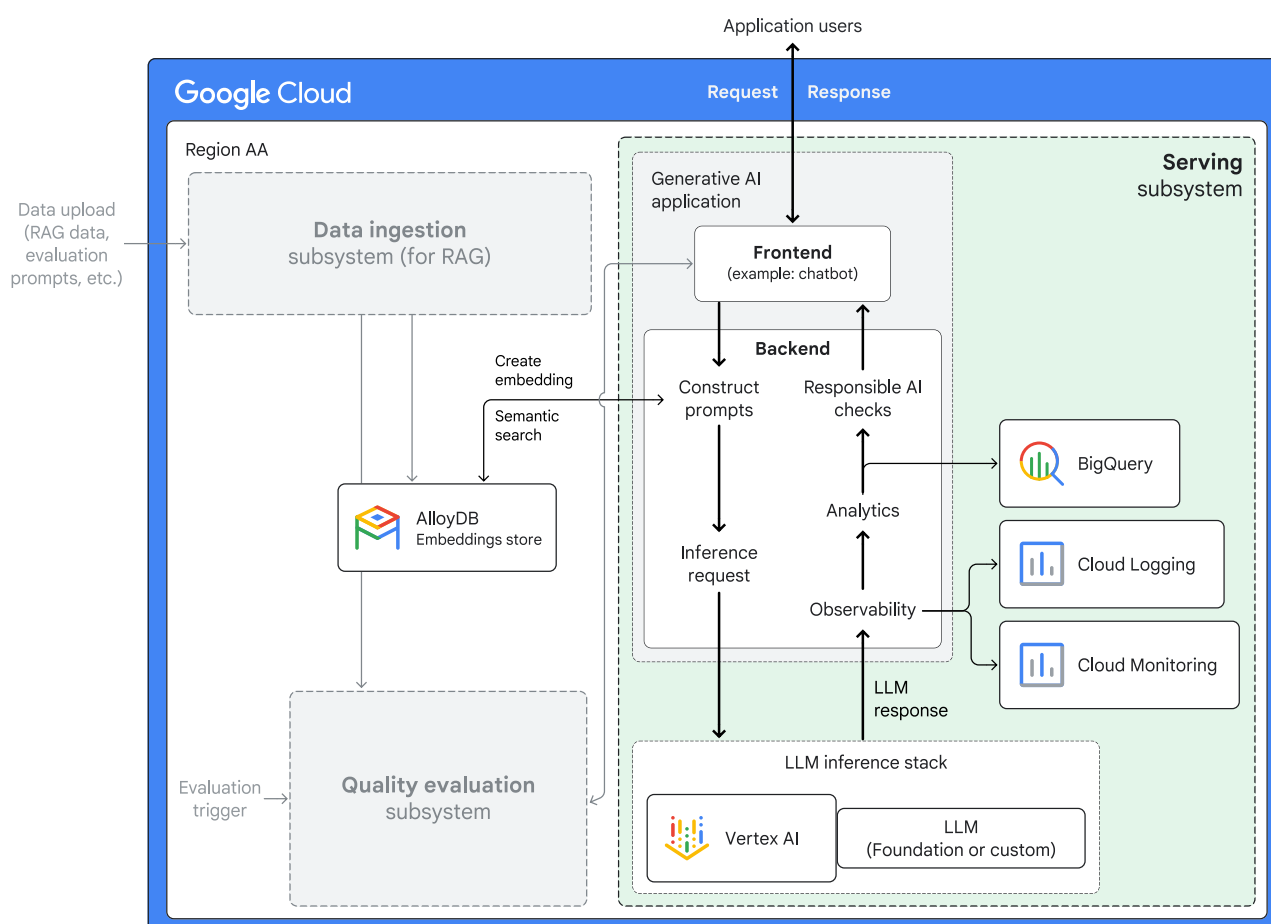
1. Data is uploaded to a Cloud Storage bucket (/storage/docs/buckets). The data sources might be applications, databases, or streaming services.
2. When data is uploaded Cloud Storage, a message is published to a Pub/Sub topic (/pubsub/docs/publish-message-overview).
3. Pub/Sub triggers a Cloud Run function (/functions/docs/concepts/overview) to process the uploaded data.
4. The function is started by using configuration data that's stored in an AlloyDB for PostgreSQL database.
5. The function parses the raw data, formats it as required, and then divides it into chunks.
6. The function uses an appropriate embedding model (/vertex-ai/generative-ai/docs/embeddings) on Vertex AI to create vectorized embeddings (/vertex-ai/docs/generative-ai/model-reference/text-embeddings) of the ingested data.

★ **Note:** The generative AI application that you develop should use the same embeddings model and parameters to convert the natural-language requests to embeddings.

7. The function stores the embeddings in an AlloyDB for PostgreSQL database that has the **pgvector** (/alloydb/docs/ai/work-with-embeddings) extension enabled. As described in the following section, when the serving subsystem processes user requests, it uses the embeddings in the vector database to retrieve relevant domain-specific data.

Serving subsystem

The serving subsystem handles the request-response flow between the generative AI application and its users. The following diagram shows details of the serving subsystem in the architecture:



The following are the steps in the request-response flow in the serving subsystem:

1. Users submit requests to the generative AI application through a frontend (for example, a chatbot or mobile app).
2. The generative AI application converts the natural-language request to embeddings (/vertex-ai/docs/glossary#embedding).

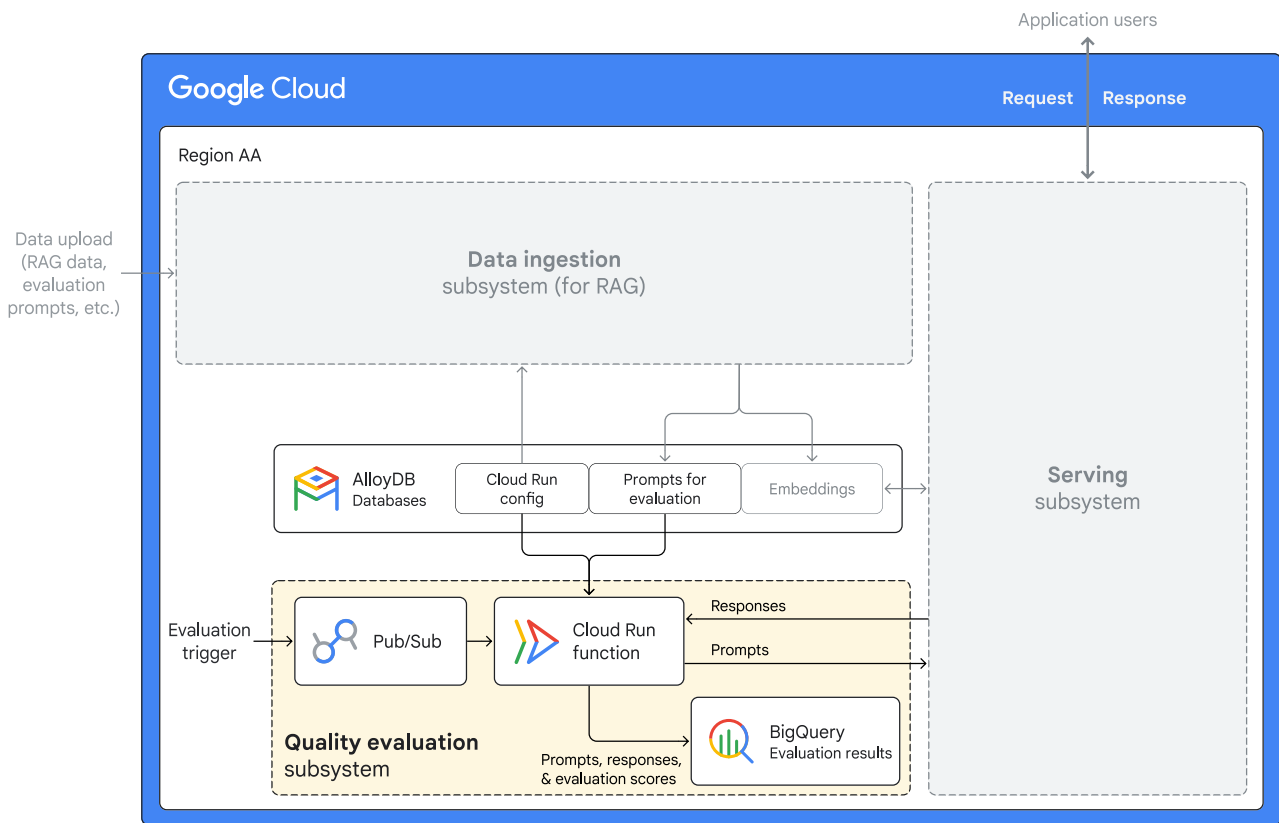
★ **Note:** To create the embeddings, the generative AI application that you develop should use the same embeddings model and parameters that are used to convert the external data to

embeddings.

3. The application completes the retrieval part of the RAG approach:
 - a. The application performs a semantic search for the embedding in the AlloyDB for PostgreSQL vector store that's maintained by the data ingestion subsystem. Semantic search helps find embeddings based on the intent of a prompt rather than its textual content.
 - b. The application combines the original request with the raw data that's retrieved based on the matching embedding to create a contextualized prompt.
4. The application sends the contextualized prompt to an LLM inference stack that runs on Vertex AI.
5. The LLM inference stack uses a generative AI LLM, which can be a foundation LLM (/vertex-ai/docs/generative-ai/learn/models#foundation_model_apis) or a custom LLM, and generates a response that's constrained to the provided context.
 - a. The application can store logs of the request-response activity in Cloud Logging. You can view and use the logs for monitoring using Cloud Monitoring. Google doesn't access or use log data.
 - b. The application loads the responses to BigQuery for offline analytics.
6. The application screens the responses by using responsible AI filters (</vertex-ai/docs/generative-ai/learn/responsible-ai>).
7. The application sends the screened responses to users through the frontend.

Quality evaluation subsystem

The following diagram shows details of the quality evaluation subsystem in the architecture:



When the quality evaluation subsystem receives a request, it does the following:

1. Pub/Sub triggers a Cloud Run function.
2. The function is started by using configuration data that's stored in an AlloyDB for PostgreSQL database.
3. The function pulls evaluation prompts from an AlloyDB for PostgreSQL database. The prompts were previously uploaded to the database by the data ingestion subsystem.
4. The function uses the evaluation prompts to assess the quality of the responses that the serving subsystem generates.

The output of this evaluation consists of evaluation scores for metrics like factual accuracy and relevance.

5. The function loads the evaluation scores and the prompts and responses that were evaluated to BigQuery for future analysis.

Products used

The following is a summary of all the Google Cloud products that the preceding architecture uses:

- [Vertex AI](https://cloud.google.com/vertex-ai) (<https://cloud.google.com/vertex-ai>): An ML platform that lets you train and deploy ML models and AI applications, and customize LLMs for use in AI-powered applications.
- [Cloud Run functions](https://cloud.google.com/functions) (<https://cloud.google.com/functions>): A serverless compute platform that lets you run single-purpose functions directly in Google Cloud.
- [BigQuery](https://cloud.google.com/bigquery) (<https://cloud.google.com/bigquery>): An enterprise data warehouse that helps you manage and analyze your data with built-in features like machine learning geospatial analysis, and business intelligence.
- [Cloud Storage](https://cloud.google.com/storage) (<https://cloud.google.com/storage>): A low-cost, no-limit object store for diverse data types. Data can be accessed from within and outside Google Cloud, and it's replicated across locations for redundancy.
- [AlloyDB for PostgreSQL](https://cloud.google.com/alloydb) (<https://cloud.google.com/alloydb>): A fully managed, PostgreSQL-compatible database service that's designed for your most demanding workloads, including hybrid transactional and analytical processing.
- [Pub/Sub](https://cloud.google.com/pubsub) (<https://cloud.google.com/pubsub>): An asynchronous and scalable messaging service that decouples services that produce messages from services that process those messages.
- [Cloud Logging](https://cloud.google.com/logging) (<https://cloud.google.com/logging>): A real-time log management system with storage, search, analysis, and alerting.
- [Cloud Monitoring](https://cloud.google.com/monitoring) (<https://cloud.google.com/monitoring>): A service that provides visibility into the performance, availability, and health of your applications and infrastructure.

Use cases

RAG is an effective technique to improve the quality of output that's generated from an LLM. This section provides examples of use cases for which you can use RAG-capable generative AI applications.

Personalized product recommendations

An online shopping site might use an LLM-powered chatbot to assist customers with finding products or getting shopping-related help. The questions from a user can be augmented by using historical data about the user's buying behavior and website interaction patterns. The data might include user reviews and feedback that's stored in an unstructured datastore or search-related metrics that are stored in a web analytics data

warehouse. The augmented question can then be processed by the LLM to generate personalized responses that the user might find more appealing and compelling.

Clinical assistance systems

Doctors in hospitals need to quickly analyze and diagnose a patient's health condition to make decisions about appropriate care and medication. A generative AI application that uses a medical LLM like Med-PaLM (<https://sites.research.google/med-palm/>) can be used to assist doctors in their clinical diagnosis process. The responses that the application generates can be grounded in historical patient records by contextualizing the doctors' prompts with data from the hospital's electronic health record (EHR) database or from an external knowledge base like PubMed (<https://pubmed.ncbi.nlm.nih.gov/>).

Efficient legal research

Generative AI-powered legal research lets lawyers quickly query large volumes of statutes and case laws to identify relevant legal precedents or summarize complex legal concepts. The output of such research can be enhanced by augmenting a lawyer's prompts with data that's retrieved from the law firm's proprietary corpus of contracts, past legal communication, and internal case records. This design approach ensures that the generated responses are relevant to the legal domain that the lawyer specializes in.

Design alternatives

This section presents alternative design approaches that you can consider for your RAG-capable generative AI application in Google Cloud.

Fully managed vector search

If you need an architecture that uses a fully managed vector search product, you can use Vertex AI and Vector Search, which provides optimized serving infrastructure for very large-scale vector search. For more information, see RAG infrastructure for generative AI using Vertex AI and Vector Search (</architecture/gen-ai-rag-vertex-ai-vector-search>).

Open-source tools and models

If you want to rapidly build and deploy RAG-capable generative AI applications by using open source tools and models Ray, Hugging Face, and LangChain, see RAG infrastructure

for generative AI using [GKE and Cloud SQL](#) (/architecture/rag-capable-gen-ai-app-using-gke).

Other RAG architectures

For information about additional architecture options for RAG workloads, see [Generative AI with RAG](#) (/architecture/rag-reference-architectures).

Design considerations

This section provides guidance to help you develop a RAG-capable generative AI architecture in Google Cloud that meets your specific requirements for security and compliance, reliability, cost, and performance. The guidance in this section isn't exhaustive. Depending on the specific requirements of your generative AI application and the Google Cloud products and features that you use, you might need to consider additional design factors and trade-offs.

Security and compliance

This section describes factors that you should consider when you design and build a RAG-capable generative AI application in Google Cloud that meets your security and compliance requirements.

Product	Design considerations
Vertex AI	Vertex AI supports Google Cloud security controls that you can use to meet your requirements for data residency, data encryption, network security, and access transparency. For more information, see Security controls for Vertex AI (/vertex-ai/docs/general/vertexai-security-controls) and Security controls for Generative AI (/vertex-ai/docs/generative-ai/genai-security-controls).
Cloud Run functions	To protect your data by using a key that you control, you can use customer-managed encryption keys (CMEK). For more information, see Using customer managed encryption keys (/functions/docs/securing/cmek).
AlloyDB for PostgreSQL	By default, data that's stored in AlloyDB for PostgreSQL is encrypted using Google-owned and Google-managed encryption keys. If you need to use encryption keys that you control and manage, you can use CMEKs. For more information, see About CMEK (/alloydb/docs/cmek). To mitigate the risk of data exfiltration from AlloyDB for PostgreSQL databases, you can create a service perimeter by using VPC Service Controls (/alloydb/docs/vpc-sc/configure-vpc-service-controls).

Product	Design considerations
	By default, an AlloyDB for PostgreSQL instance accepts only connections that use SSL. To further secure connections to your AlloyDB for PostgreSQL databases, you can use the AlloyDB for PostgreSQL Auth Proxy connector. The Auth Proxy connector provides Identity and Access Management (IAM)-based connection authorization and uses a TLS 1.3 connection with a 256-bit AES cipher to verify client and server identities and encrypt data traffic. For more information, see About the AlloyDB for PostgreSQL Auth Proxy (/alloydb/docs/auth-proxy/overview). For connections created by using Java, Python, or Go, use the appropriate Language Connector (/alloydb/docs/language-connectors-overview) instead of the Auth Proxy connector. AlloyDB for PostgreSQL helps you meet data residency requirements. Data is stored or replicated within the regions that you specify.
BigQuery	BigQuery provides many features that you can use to control access to data, protect sensitive data, and ensure data accuracy and consistency. For more information, see Introduction to data governance in BigQuery (/bigquery/docs/data-governance). BigQuery helps you meet data residency requirements. Data is stored within the region that you specify.
Cloud Storage	By default, the data that's stored in Cloud Storage is encrypted using Google-owned and Google-managed encryption keys. If required, you can use CMEKs or your own keys that you manage by using an external management method like customer-supplied encryption keys (CSEKs). For more information, see Data encryption options (/storage/docs/encryption). Cloud Storage supports two methods for granting users access to your buckets and objects: IAM and access control lists (ACLs). In most cases, we recommend using IAM, which lets you grant permissions at the bucket and project levels. For more information, see Overview of access control (/storage/docs/access-control). The data that you load into the data ingestion subsystem through Cloud Storage might include sensitive data. To protect such data, you can use Sensitive Data Protection to discover, classify, and de-identify the data. For more information, see Using Sensitive Data Protection with Cloud Storage (/sensitive-data-protection/docs/dlp-gcs). Cloud Storage helps you meet data residency requirements. Data is stored or replicated within the regions that you specify.
Pub/Sub	By default, Pub/Sub encrypts all messages, both at rest and in transit, by using Google-owned and Google-managed encryption keys. Pub/Sub supports the use of CMEKs for message encryption at the application layer. For more information, see Configuring message encryption (/pubsub/docs/encryption). If you have data residency requirements, to ensure that message data is stored in specific locations, you can configure message storage policies (/pubsub/docs/resource-location-restriction).

Product	Design considerations
Cloud Logging	Admin Activity audit logs (/logging/docs/audit#admin-activity) are enabled by default for all the Google Cloud services that are used in this reference architecture. These logs record API calls or other actions that modify the configuration or metadata of Google Cloud resources. Data Access audit logs (/logging/docs/audit#data-access) are enabled by default for BigQuery. For the other services that are used in this architecture, you can enable Data Access audit logs. The logs let you track API calls that read the configuration or metadata of resources or user requests to create, modify, or read user-provided resource data. To help meet data residency requirements, you can configure Cloud Logging to store log data in the region that you specify. For more information, see Regionalize your logs (/logging/docs/regionalized-logs).

For security principles and recommendations that are specific to AI and ML workloads, see [AI and ML perspective: Security](#) (/architecture/framework/perspectives/ai-ml/security) in the Well-Architected Framework.

Reliability

This section describes design factors that you should consider to build and operate reliable infrastructure for a RAG-capable generative AI application in Google Cloud.

Product	Design considerations
AlloyDB for PostgreSQL	By default, AlloyDB for PostgreSQL clusters provide high availability (HA) with automatic failover. The primary instance has redundant nodes that are located in two different zones within a region. This redundancy ensures that the clusters are robust against zone outages. To plan for recovery from region outages, you can use cross-region replication (/alloydb/docs/cross-region-replication/about-cross-region-replication).
BigQuery	Data that you load into BigQuery is stored synchronously in two zones within the region that you specify. This redundancy helps ensure that your data isn't lost when a zone outage occurs. For more information about reliability features in BigQuery, see Understand reliability (/bigquery/docs/reliability-intro).
Cloud Storage	You can create Cloud Storage buckets in one of three location types (/storage/docs/locations): regional, dual-region, or multi-region. Data stored in regional buckets is replicated synchronously across multiple zones within a region. For higher

Product	Design considerations
	availability, you can use dual-region or multi-region buckets, where data is replicated asynchronously across regions.
Pub/Sub	To manage transient spikes in message traffic, you can configure flow control (/pubsub/docs/flow-control-messages) in the publisher settings. To handle failed publishes, adjust the retry-request variables as necessary. For more information, see Retry requests (/pubsub/docs/retry-requests).

Note: For more information about region-specific considerations, see [Geography and regions](#) (/docs/geography-and-regions#regions_and_zones).

For reliability principles and recommendations that are specific to AI and ML workloads, see [AI and ML perspective: Reliability](#) (/architecture/framework/perspectives/ai-ml/reliability) in the Well-Architected Framework.

Cost optimization

This section provides guidance to help you optimize the cost of setting up and operating a RAG-capable generative AI application in Google Cloud.

Product	Design considerations
AlloyDB for PostgreSQL	By default, a primary instance of an AlloyDB for PostgreSQL cluster is highly available (HA). The instance has an active node and a standby node. If the active node fails, AlloyDB for PostgreSQL fails over to the standby node automatically. If you don't need HA for the databases, then you can reduce cost by making the cluster's primary instance a basic instance. A basic instance isn't robust against zone outages and it has longer downtime during maintenance operations. For more information, see Reduce costs using basic instances (/alloydb/docs/basic-instance). If you can predict the CPU and memory requirements of your AlloyDB for PostgreSQL instance, then you can save money by getting discounts for committed usage. For more information, see AlloyDB for PostgreSQL committed use discounts (/alloydb/cud).
BigQuery	BigQuery lets you estimate the cost of queries before running them. To optimize query costs, you need to optimize storage and query computation. For more information, see Estimate and control costs (/bigquery/docs/best-practices-costs).
Cloud Storage	For the Cloud Storage bucket that you use to load data into the data ingestion subsystem, choose an appropriate storage class (/storage/docs/storage-classes) based on the data-retention and access-frequency requirements of your workloads. For example, you can choose the Standard storage class, and use Object Lifecycle Management

Product	Design considerations
	(/storage/docs/lifecycle) to control storage costs by automatically downgrading objects to a lower-cost storage class or deleting objects based on conditions that you set.
Cloud Logging	To control the cost of storing logs, you can do the following: - Reduce the volume of logs by excluding or filtering unnecessary log entries. For more information, see Exclusion filters (/logging/docs/exclusions). - Reduce the period for which log entries are retained. For more information, see Configure custom retention (/logging/docs/buckets#custom-retention).

For cost optimization principles and recommendations that are specific to AI and ML workloads, see [AI and ML perspective: Cost optimization](/architecture/framework/perspectives/ai-ml/cost-optimization) (/architecture/framework/perspectives/ai-ml/cost-optimization) in the Well-Architected Framework.

Performance

This section describes the factors that you should consider when you design and build a RAG-capable generative AI application in Google Cloud that meets your performance requirements.

Product	Design considerations
AlloyDB for PostgreSQL	To help you analyze and improve query performance of the databases, AlloyDB for PostgreSQL provides a Query Insights tool. You can use this tool to monitor performance and trace the source of a problematic query. For more information, see Query Insights overview (/alloydb/docs/query-insights-overview). To get an overview of the status and performance of your databases and to view detailed metrics such as peak connections and maximum replication lag, you can use the System Insights dashboard. For more information, see Monitor an instance using the AlloyDB for PostgreSQL System Insights dashboard (/alloydb/docs/monitor-instance#monitoring). To reduce the load on your primary AlloyDB for PostgreSQL instance and to scale out the capacity to handle read requests, you can add read pool instances to the cluster. For more information, see AlloyDB for PostgreSQL nodes and instances (/alloydb/docs/overview#instances).
BigQuery	BigQuery provides a query execution graph that you can use to analyze query performance and get performance insights for issues like slot contention and insufficient shuffle quota. For more information, see Get query performance insights (/bigquery/docs/query-insights). After you address the issues that you identify through query performance insights, you can further optimize queries by using techniques like reducing the volume of input and output

Product	Design considerations
	data. For more information, see Optimize query computation (/bigquery/docs/best-practices-performance-compute).
Cloud Storage	To upload large files, you can use a method called <i>parallel composite uploads</i> . With this strategy, the large file is split into chunks. The chunks are uploaded to Cloud Storage in parallel and then the data is recomposed in the cloud. Parallel composite uploads can be faster than regular upload operations when network bandwidth and disk speed aren't limiting factors. However, this strategy has some limitations and cost implications. For more information, see Parallel composite uploads (/storage/docs/parallel-composite-uploads).

For performance optimization principles and recommendations that are specific to AI and ML workloads, see [AI and ML perspective: Performance optimization](/architecture/framework/perspectives/ai-ml/performance-optimization) (/architecture/framework/perspectives/ai-ml/performance-optimization) in the Well-Architected Framework.

Deployment

To get started and experiment with building infrastructure on Google Cloud for RAG-capable generative AI applications, you can use [Jump Start Solution: Generative AI RAG with Cloud SQL](/architecture/ai-ml/generative-ai-rag) (/architecture/ai-ml/generative-ai-rag). This solution deploys a Python-based chat application on Cloud Run and uses a fully managed Cloud SQL database for vector search. The sample code for this solution is available in [GitHub](https://github.com/GoogleCloudPlatform/terraform-genai-rag) (https://github.com/GoogleCloudPlatform/terraform-genai-rag).

Note: The Jump Start Solution uses Cloud SQL as the vector database, whereas this reference architecture uses AlloyDB for PostgreSQL. Both Cloud SQL and AlloyDB for PostgreSQL support the [pgvector extension](https://github.com/pgvector/pgvector) (https://github.com/pgvector/pgvector), which lets you run vector search in a PostgreSQL database.

What's next

- [Choose models and infrastructure for your generative AI application](/docs/generative-ai/choose-models-infra-for-ai) (/docs/generative-ai/choose-models-infra-for-ai)
- [Build an LLM and RAG-based chat application using AlloyDB for PostgreSQL and LangChain](https://codelabs.developers.google.com/codelabs/genai-db-retrieval-app#0) (https://codelabs.developers.google.com/codelabs/genai-db-retrieval-app#0)

- [RAG infrastructure for generative AI using Vertex AI and Vector Search](/architecture/gen-ai-rag-vertex-ai-vector-search)
(/architecture/gen-ai-rag-vertex-ai-vector-search)
- [RAG infrastructure for generative AI using GKE and Cloud SQL](/architecture/rag-capable-gen-ai-app-using-gke)
(/architecture/rag-capable-gen-ai-app-using-gke)
- [RAG infrastructure for generative AI using Gemini Enterprise and Vertex AI](/architecture/rag-genai-agentspace-vertexai)
(/architecture/rag-genai-agentspace-vertexai)
- [GraphRAG infrastructure for generative AI using Vertex AI and Spanner Graph](/architecture/gen-ai-graphrag-spanner)
(/architecture/gen-ai-graphrag-spanner)
- For an overview of architectural principles and recommendations that are specific to AI and ML workloads in Google Cloud, see the [AI and ML perspective](/architecture/framework/perspectives/ai-ml)
(/architecture/framework/perspectives/ai-ml) in the Well-Architected Framework.
- For more reference architectures, diagrams, and best practices, explore the [Cloud Architecture Center](/architecture) (/architecture).

Contributors

Author: [Kumar Dhanagopal](https://www.linkedin.com/in/kumardhanagopal) (https://www.linkedin.com/in/kumardhanagopal) | Cross-Product Solution Developer

Other contributors:

- [Andrew Brook](https://www.linkedin.com/in/andrewbrook) (https://www.linkedin.com/in/andrewbrook) | Engineering Director
- [Anna Berenberg](https://www.linkedin.com/in/annaberenberg) (https://www.linkedin.com/in/annaberenberg) | Engineering Fellow
- [Assaf Namer](https://www.linkedin.com/in/assaf-namer-0677644) (https://www.linkedin.com/in/assaf-namer-0677644) | Principal Cloud Security Architect
- [Balachandar Krishnamoorthy](https://www.linkedin.com/in/balachandar-k-362a3918) (https://www.linkedin.com/in/balachandar-k-362a3918) | Principal Software Engineer
- [Daniel Lees](https://www.linkedin.com/in/daniellees) (https://www.linkedin.com/in/daniellees) | Cloud Security Architect
- [Derek Downey](https://www.linkedin.com/in/derekjdowney) (https://www.linkedin.com/in/derekjdowney) | Developer Relations Engineer
- [Eran Lewis](https://www.linkedin.com/in/eranlewis) (https://www.linkedin.com/in/eranlewis) | Senior Product Manager
- [Geoffrey Anderson](https://www.linkedin.com/in/geoffsdesk) (https://www.linkedin.com/in/geoffsdesk) | Product Manager
- [Gleb Otochkin](https://www.linkedin.com/in/glebotochkin) (https://www.linkedin.com/in/glebotochkin) | Cloud Advocate, Databases

- [Hamsa Buvaraghan](https://www.linkedin.com/in/hamsa-buvaraghan-9911bb8) (https://www.linkedin.com/in/hamsa-buvaraghan-9911bb8) | AI Product Manager
- [Irina Sigler](https://www.linkedin.com/in/irina-sigler-298a59b1) (https://www.linkedin.com/in/irina-sigler-298a59b1) | Product Manager
- [Jack Wotherspoon](https://www.linkedin.com/in/jack-wotherspoon) (https://www.linkedin.com/in/jack-wotherspoon) | Developer Advocate
- [Jason Davenport](https://www.linkedin.com/in/davenportjw) (https://www.linkedin.com/in/davenportjw) | Developer Advocate
- [Jordan Totten](https://www.linkedin.com/in/jordantotten) (https://www.linkedin.com/in/jordantotten) | Customer Engineer
- [Julia Wiesinger](https://www.linkedin.com/in/julia-eschelbeck) (https://www.linkedin.com/in/julia-eschelbeck) | Product Manager
- [Kara Greenfield](https://www.linkedin.com/in/kara-greenfield-988b2885) (https://www.linkedin.com/in/kara-greenfield-988b2885) | Customer Engineer
- [Kurtis Van Gent](https://www.linkedin.com/in/kurtisvg) (https://www.linkedin.com/in/kurtisvg) | Staff Software Engineer
- [Per Jacobsson](https://www.linkedin.com/in/per-jacobsson-20b9613) (https://www.linkedin.com/in/per-jacobsson-20b9613) | Software Engineer
- [Pranav Nambiar](https://www.linkedin.com/in/pranavnambiar) (https://www.linkedin.com/in/pranavnambiar) | Director
- Richard Hendricks | Architecture Center Staff
- [Safiuddin Khaja](https://www.linkedin.com/in/safiuddink) (https://www.linkedin.com/in/safiuddink) | Cloud Engineer
- [Sandy Ghai](https://www.linkedin.com/in/sandyghai) (https://www.linkedin.com/in/sandyghai) | Group Product Manager
- [Vladimir Vuskovic](https://www.linkedin.com/in/vladimir-vuskovic-626595) (https://www.linkedin.com/in/vladimir-vuskovic-626595) | Product Management Director
- [Stereon Giannini](https://www.linkedin.com/in/steren) (https://www.linkedin.com/in/steren) | Group Product Manager
- [Wietse Venema](https://www.linkedin.com/in/wietse-venema) (https://www.linkedin.com/in/wietse-venema) | Developer Relations Engineer

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (https://www.apache.org/licenses/LICENSE-2.0). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-02-04 UTC.